

Astro 142

Discussion 14

2025-11-18
Dyson Lewis

Discussion Activity

Discussion 14 Activity

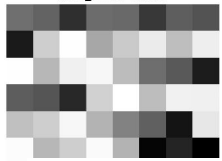
1. Download the data and make a visualization of the eigenvalues and eigenvectors. Try to reconstruct with the first few eigenvectors.
2. Follow the tutorial here:
<https://emcee.readthedocs.io/en/latest/tutorials/quickstart/>

Paste screenshots into the slides. Upload your final pdf to the discussion assignment

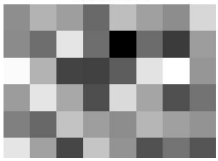
If you have extra time, work on the homework! Ask questions!

Face Reconstruction Comparison (Face #0)

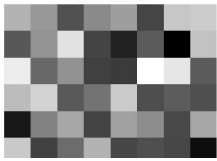
Original Face



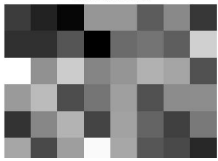
1 Components
MSE: 0.0683
Var: 54.6%



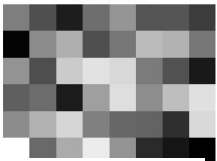
2 Components
MSE: 0.0632
Var: 63.9%



5 Components
MSE: 0.0473
Var: 79.4%



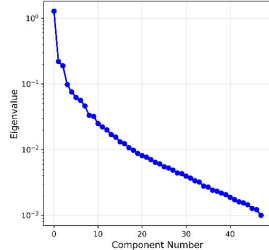
10 Components
MSE: 0.0132
Var: 89.2%



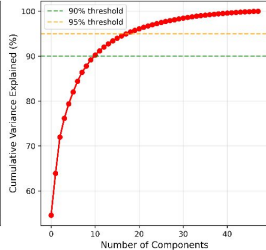
20 Components
MSE: 0.0033
Var: 95.7%



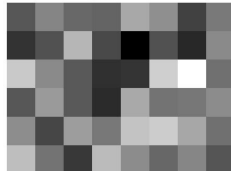
Eigenvalue Spectrum (Scree Plot)



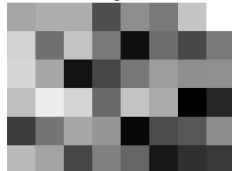
Cumulative Explained Variance



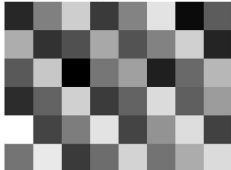
Mean Face



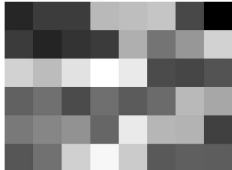
1st Eigenface



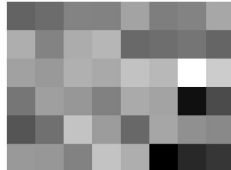
Eigenface 2



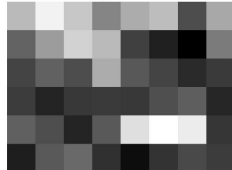
Eigenface 3



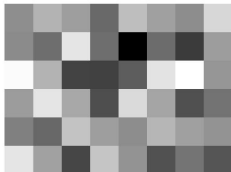
Eigenface 4



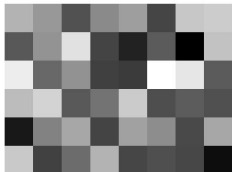
Eigenface 5



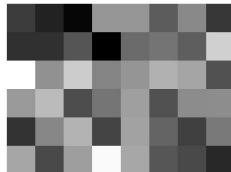
1 Components
MSE: 0.0683
Var: 54.6%



2 Components
MSE: 0.0632
Var: 63.9%



5 Components
MSE: 0.0473
Var: 79.4%



10 Components
MSE: 0.0132
Var: 89.2%



```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.io import loadmat
4 import os
5
6 # Get the directory where this script is located
7 script_dir = os.path.dirname(os.path.abspath(__file__))
8 mat_file = os.path.join(script_dir, 'demo_data.mat')
9
10 # Load the .mat file
11 print(f"Loading data from {mat_file}...")
12 data = loadmat(mat_file)
13
14 # Print available variables in the .mat file
15 print("\nAvailable variables in .mat file:")
16 for key in data.keys():
17     if not key.startswith('__'):
18         print(f"    {key}: shape {data[key].shape}")
19
20 # Extract the faces data
21 X = data['faces']
22 print(f"\nUsing data matrix: faces")
23 print(f>Data shape: {X.shape}")
24 print(f"Number of face images: {X.shape[0]}")
25 print(f"Pixels per face: {X.shape[1]} (likely 48x75 or similar)")
26
27 # Center the data
28 X_mean = np.mean(X, axis=0)
29 X_centered = X - X_mean
30
31 # Compute covariance matrix and eigenvalues/eigenvectors
32 cov_matrix = np.cov(X_centered.T)
33 eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)
34
35 # Sort eigenvalues and eigenvectors in descending order
36 idx = eigenvalues.argsort()[::-1]
37 eigenvalues = eigenvalues[idx]
38 eigenvectors = eigenvectors[:, idx]
39
40 # Create visualizations
41 fig = plt.figure(figsize=(20, 14))

```

```

43 # 1. Eigenvalue spectrum
44 ax1 = plt.subplot(3, 4, 1)
45 plt.plot(eigenvalues, 'bo-', linewidth=2, markersize=6)
46 plt.xlabel('Component Number', fontsize=12)
47 plt.ylabel('Eigenvalue', fontsize=12)
48 plt.title('Eigenvalue Spectrum (Scree Plot)', fontsize=14, fontweight='bold')
49 plt.grid(True, alpha=0.3)
50 plt.yscale('log')
51
52 # 2. Cumulative explained variance
53 ax2 = plt.subplot(3, 4, 2)
54 explained_var = eigenvalues / np.sum(eigenvalues) * 100
55 cumulative_var = np.cumsum(explained_var)
56 plt.plot(cumulative_var, 'ro-', linewidth=2, markersize=6)
57 plt.xlabel('Number of Components', fontsize=12)
58 plt.ylabel('Cumulative Variance Explained (%)', fontsize=12)
59 plt.title('Cumulative Explained Variance', fontsize=14, fontweight='bold')
60 plt.grid(True, alpha=0.3)
61 plt.axhline(y=90, color='g', linestyle='--', label='90% threshold', alpha=0.7)
62 plt.axhline(y=95, color='orange', linestyle='--', label='95% threshold', alpha=0.7)
63 plt.legend()
64
65 # 3. Eigenface visualization (mean face)
66 ax3 = plt.subplot(3, 4, 3)
67 # Assume 48 pixels might be 6x8 or 8x6
68 img_h, img_w = 6, 8
69 mean_face = X_mean.reshape(img_h, img_w)
70 plt.imshow(mean_face, cmap='gray', interpolation='nearest')
71 plt.title('Mean Face', fontsize=14, fontweight='bold')
72 plt.axis('off')
73
74 # 4. First eigenvector as eigenface
75 ax4 = plt.subplot(3, 4, 4)
76 eigenface_1 = eigenvectors[:, 0].reshape(img_h, img_w)
77 plt.imshow(eigenface_1, cmap='gray', interpolation='nearest')
78 plt.title('1st Eigenface', fontsize=14, fontweight='bold')
79 plt.axis('off')
80

```